In the Age of Agentic, the CPU is the New Bottleneck

guanlan

2026-03-27 2280 words 11 minutes

For the last three years, the center of gravity in AI felt obvious. Bigger models. More GPUs. Faster inference. Almost every serious conversation in the industry eventually collapsed back to the same axis.

Walking into SAP Center, I expected Jensen Huang to keep pushing the familiar story: more tokens, more throughput, more chips. Instead, he put Vera Rubin at the center of the stage.

What stood out was the framing. The pitch was no longer that AI simply needs more compute. It was that once AI starts behaving like agents, the hard part is no longer the model alone. It is the system around the

components.

In that world, the CPU matters again.

A chatbot has a clean loop. A user sends a prompt, the model runs, and a response comes back. In that flow, the GPU is usually the bottleneck, and the rest of the system mostly exists to keep the GPU busy.

A production agent is different. It takes a task, pulls context, calls the model, parses the output, decides what to do next, invokes a tool, waits on I/O, handles the result, prepares the next prompt, and calls the model again. Then it repeats. Sometimes a few times. Sometimes hundreds.

Once that loop becomes the real workload, model inference is no longer the whole story. It becomes one stage inside a larger execution cycle. Everything around it starts to matter: tool execution, retrieval, serialization, scheduling, sandboxing, retries, context assembly, and state persistence. Most of that work lives on the CPU.

The old bottleneck story is starting to break

We are starting to see it in the data.

Late last year, Georgia Tech and Intel published a paper (A CPU-Centric Perspective on Agentic AI, arXiv:2511.00739) that profiled five typical agent workflows. Their conclusion: tool processing on the CPU accounts for 50% to 90.6% of total latency. In RAG scenarios, retrieval is the bottleneck. In coding agents, executing bash and Python is the bottleneck. In research agents, waiting for and parsing literature search APIs is the bottleneck.

vLLM's own data paints an even starker picture. When running Llama 3 8B on a single H100 GPU, HTTP services accounted for 33% of execution time, and scheduling plus input preparation took 29%. Actual GPU computation was only 38%. With the Blackwell generation making GPUs even faster, this has only gotten worse. As vLLM engineers put it: the GPU is simply too fast, and the CPU cannot feed it data quickly enough. They recently optimized HTTP/gRPC serialization and got a 57% performance boost at 1024 concurrency. That entire 57% gap had been eaten by CPU-side serialization overhead.

This is the most noteworthy shift at GTC this year. NVIDIA is no longer framing the problem simply as models needing more GPUs. They are emphasizing execution-layer components: agentic responses, runtime engines, orchestration services, tooling, and analytics pipelines. The product narrative behind Vera is that the future value of AI systems will come not just from faster compute, but from highly efficient execution synergy between the CPU and GPU.

Why the logic of the agent era is different

architecture. Agent execution logic is entirely different.

An agent is a long-running state machine. Every step involves conditional logic, branching, external I/O, and state persistence. These operations are serial, I/O-bound, and involve moving large amounts of small data rather than massive matrix math. This is what the CPU is built for, and also where it easily bottlenecks.

Think of it through Amdahl's Law . As GPU inference speeds up by an order of magnitude, the percentage of time spent on GPU compute in the overall pipeline shrinks. The remaining serial tasks, CPU scheduling, data preprocessing, tool calling, and context management, become the dominant source of latency. The faster the GPU gets, the more obvious the CPU bottleneck becomes.

Software architecture amplifies this. Python-based agent frameworks like LangChain and LlamaIndex consume 5x more memory, use 2-3x more CPU, and have cold starts 14-34x slower than their Rust-based equivalents (per Saivishwak's benchmark). Multi-agent systems are worse. Token duplication rates across mainstream frameworks sit between 53% and 86%, and coordination overhead grows at $O(n^2)$ with the number of agents. O'Reilly's analysis shows that a multi-agent system consumes roughly 15 times the tokens of a single agent, with most of that budget going to coordination, not inference.

The GPU is the thinking engine. The CPU is the execution and scheduling hub. The industry has bet almost everything on the thinking engine, but the scheduling hub is dragging the system down.

What Vera is actually betting on

Seen through that lens, Vera starts to make a lot more sense.

On paper, some of the raw specs can look counterintuitive. Vera has 88 custom Olympus cores built on ARM v9.2, using TSMC 3nm, on a monolithic die. That is fewer cores than AMD Turin's 192 or Intel Granite Rapids' 128.

But the point is not maximal core count.

NVIDIA is claiming around 1.5x the IPC of Grace's Neoverse V2, which would put Vera far ahead on single-thread performance. That matters because agent workloads don't behave like neat, embarrassingly parallel cloud jobs. They spend a lot of time in branchy control flow, waiting on tools, serializing state, handling I/O, and preparing the next step. In those moments, the question is how fast one thread can move when the rest of the pipeline is waiting on it.

Jensen made this point clearly in his interview with Ben Thompson. Over the last decade, CPUs were largely optimized for hyperscale cloud economics. Cloud providers made money by renting out lots of cores, so

idle while a tool call crawls back through the stack, what matters is how quickly the serial path can clear.

That also helps explain Vera's monolithic die design. AMD's chiplet strategy is great for core count economics, but cross-die NUMA latency is not a friendly fit for the bursty, unpredictable behavior of agent workloads. NVIDIA is making a different tradeoff here, one that favors latency consistency over raw scale.

The same logic shows up in Spatial Multithreading. Instead of relying on traditional time-sliced hyperthreading, Vera physically isolates core resources to keep latency more deterministic. If the target world is thousands of concurrent agent runtimes, sandboxes, and tool chains, jitter starts to matter.

The memory subsystem follows the same pattern. LPDDR5X, up to 1.2 TB/s of bandwidth and 1.5 TB of capacity, works out to roughly 14 GB/s per core. That is far above typical x86 server memory bandwidth per core. It also uses about half the power of DDR5, which helps explain NVIDIA's pitch that a Vera-based rack can be far more power-efficient than a comparable x86 setup.

But the real architectural bet is NVLink-C2C.

Vera talks to the GPU over a coherent 1.8 TB/s bidirectional interconnect. That is dramatically beyond PCIe Gen5 or Gen6 in both bandwidth and latency. The CPU and GPU share unified page tables, so data can move at cache-line granularity without explicit cudaMemcpy gymnastics.

For classic AI training, that is nice. For agents, it is more than nice.

It means a KV cache can spill to CPU-side memory without turning into a copy-and-marshaling nightmare. It means tool execution results can come back and feed the next inference step with much less handoff overhead. It means the system starts to behave less like

two different compute domains.

That is the deeper idea behind Vera. Tighter coupling between reasoning and execution.

The industry is validating the bet

Intel was already positioning Xeon as the orchestration layer for AI systems, leaning into the fact that agentic workloads need CPU capacity even as GPUs stay scarce. AMD has been telling a similar story around EPYC, with Lisa Su pointing to rising demand from AI orchestration and mixed CPU-GPU deployments.

The market data is also starting to move. Server CPU prices are up. Lead times are stretching. Some projections now have the server CPU market growing from around $27B in 2025 to roughly $60B by 2030. It means the rest of the stack is finally being repriced.

Even the open-source and developer ecosystem is reflecting this.

Peter Steinberger at the pre-show

agentic computer. NVIDIA also introduced NemoClaw as a more enterprise-ready reference stack, and brought founder Peter Steinberger on stage in the pre-show.

That was a very intentional choice. OpenClaw completes the story that Vera needs. If the hardware pitch is that agent workloads deserve a purpose-built execution platform, then the software story has to show that those workloads are real, growing, and operationally painful enough to justify new infrastructure.

The Hacker News reaction was predictably split. Some people dismissed Vera as another AI-branded CPU, with agentic slapped on top because it is 2026 and everything needs the word agent in the title. But the rebuttal is not hard to see. Features like FP8 support and a 1.8 TB/s coherent CPU-GPU interconnect are direct responses to AI system behavior.

Purpose-built is the right word.

Hardware is only half the answer

Still, I don’t think the long-term opportunity here is mainly in hardware.

Vera validates the premise. It doesn’t solve the whole problem.

If vLLM can recover 57% just by fixing serialization overhead, if Python-based agent stacks are wildly less efficient than leaner runtimes, and if multi-agent frameworks are wasting huge amounts of tokens on coordination, then there is still a massive software gap sitting in plain sight.

That gap matters because the current stack was not built for agents.

Kubernetes schedules containers. It doesn’t understand long-running, stateful, partially completed agent executions. Temporal gives durable workflows, but its persistence boundaries sit at the wrong level for

not for tight control over runtime behavior, failure semantics, or effect isolation.

All of these tools are useful. None of them are the right native abstraction.

Agents need an execution layer that understands their actual semantics: state persistence, recovery after partial failure, control over real-world side effects, etc. Stitching together older infrastructure gets you part of the way, but not the whole way. It is the same mistake as bolting together a file system, a process scheduler, and a network stack and pretending an operating system has magically appeared.

The missing layer is runtime infrastructure that treats agent execution as a first-class systems problem.

Security is part of execution

Security breaks in the same place.

Once an agent can touch private data, consume untrusted inputs, and call out to the world, prompt injection stops being an edge case and starts becoming an execution problem. Most of the industry still responds to that with prompt filtering, which is nowhere near enough.

The real issue is that the trust boundary is in the wrong place.

My team built an open-source project called ClawShell (github.com/clawshell) around this idea. The basic design is simple: put a secure proxy layer between the agent and external services, and use operating-system-level boundaries to separate what the agent can see from what the system can actually do. The agent holds only virtual credentials. Real API key exchange, DLP checks, email filtering, and policy enforcement all happen inside the gateway.

ClawShell vs. OpenShell Security Architecture

After we released it, NVIDIA's investment arm reached out to talk. Then at GTC, NVIDIA introduced OpenShell inside the NemoClaw stack. Different implementation, same underlying direction: gateway-enforced trust boundaries, policy execution, strong authentication, tighter control over external effects.

I don't want to overstate the parallel. But the convergence matters.

It suggests that agent security is starting to mature in the right direction, as execution governance sitting around the model.

What Vera doesn't solve

GTC 2026 made that shift harder to ignore. NVIDIA is now designing around agent execution as a real systems workload, not as a footnote to GPU compute.

But naming the problem is not the same as solving it.

Vera addresses the hardware side. A faster CPU, a tighter interconnect, better memory bandwidth per

and the real world is still stitched together from tools that were designed for a different era, then faster hardware just means you hit the same architectural ceilings at higher clock speeds.

This is what my team is building. Our bet is that agent execution needs a small number of new primitives at the runtime layer: durable state that survives partial failure, a capability gateway that governs what the agent can actually touch, and fork-and-recover semantics that let you roll back real-world side effects when things go wrong. Not a framework. Not an orchestrator. An execution substrate that treats the agent's lifecycle as the unit of work.

I could be wrong about the specific primitives. But I am fairly confident about the shape of the problem. The gap between what agents need and what existing infrastructure provides is not going to close by itself, and it is not going to be closed by the model providers or the chip makers.

It will be closed by whoever builds the runtime that makes agent execution boring and reliable, the way Linux made process scheduling boring and reliable.

That is what we are working on.

Updated on 2026-03-27

Agent, Infra

Back | Home

< Agent Infrastructure Needs New Primitives
I Read Claude Code's Leaked Source. It Exposes the Limits of Shell-First Safety. >

Guanlan Dai | CC BY-NC 4.0
©2009 - 2026 Guanlan Dai